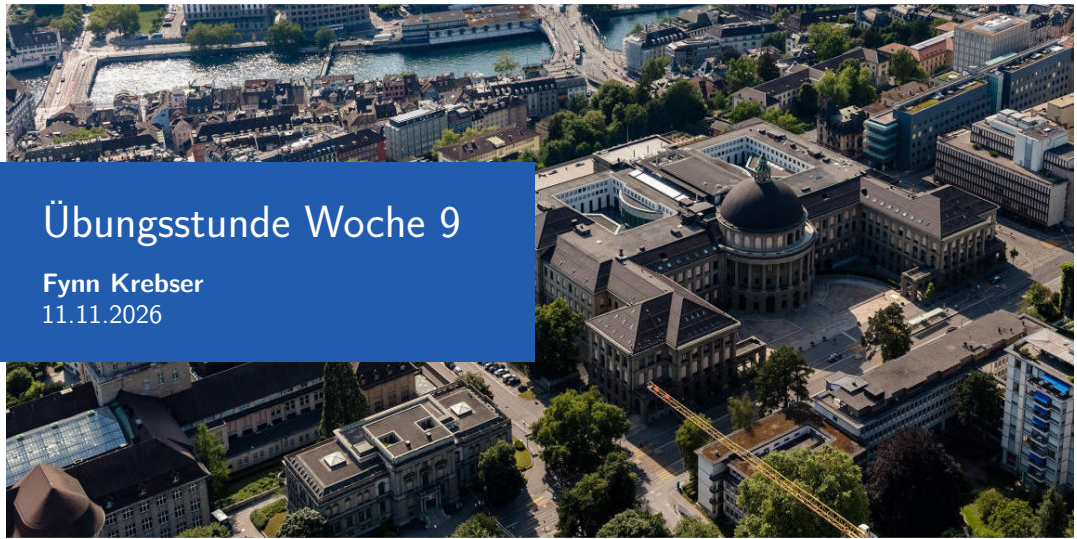
ETH zürich

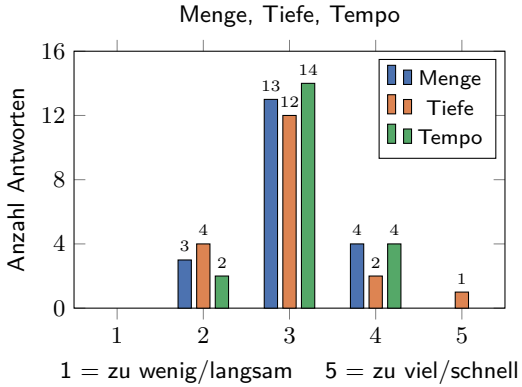


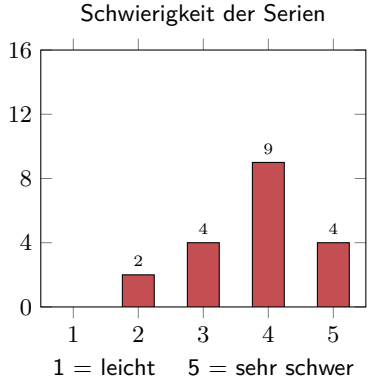
# Übungsstunde Woche 9

**Fynn Krebser**

11.11.2026

# Euer Feedback: Danke! ($n = 20$)



# Alte Prüfungsfrage

```c
int mystery(int n) {
  assert(n > 0);
  if (n > 1) {
    return n + mystery(n - 1);
  } else {
    return 1;
  }
}
```

Gegeben sei Funktion `mystery(int n)`, die mit einer positiven Zahl als Benutzereingabe in `main()` aufgerufen wird. Welches mathematische Problem berechnet die Funktion (für positive Zahlen)? / Consider function `mystery(int n)`, which is called with a positive number in `main()`. Which mathematical problem does the function compute (for positive numbers)?

- ○ Fakultät von `n` / Factorial of `n`
- ○ `n` hoch `n` / `n`-th power of `n`
- ○ Summe aller Zahlen bis `n` / Sum of all numbers up to `n`
- ○ `n`-te Fibonacci Zahl / `n`-th Fibonacci number

# Serie letzter Woche

Allgemein: Kommentare könnten vielen noch helfen...

- **Decimal to Arbitrary Base**: Gab zwei Ansaätze: Rekursiv und Iterativ. Als Grundsatz für Prüfungsfragen: In der Regel gibt es eine relativ kurze Lösung.
- **Binary Expansion**: Gute Stilübung. Für diejenigen die D&A besuchen werden solche Implementationsaufgaben noch viel vorkommen.

Fragen?

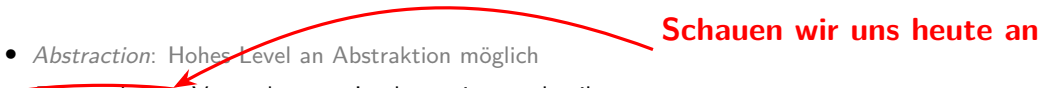
# Ziel von Heute

- Eigene Datentypen mit `struct`s
- Rekursion die Zweite

# Prinzipien des Objektorientierten Programmierens von C++

- *Abstraction*: Hohes Level an Abstraktion möglich
- *Encapsulation*: Verstecken von Implementierungsdetails
- *Inheritance*: Wiederverwendung von Code möglich
- *Polymorphism*: Gleiche Schnittstelle, unterschiedliche Implementierung

# Prinzipien des Objektorientierten Programmierens von C++




- *Abstraction*: Hohes Level an Abstraktion möglich
- *Encapsulation*: Verstecken von Implementierungsdetails
- *Inheritance*: Wiederverwendung von Code möglich
- *Polymorphism*: Gleiche Schnittstelle, unterschiedliche Implementierung

# Eigene Datentypen: Structs

- Manchmal reichen Datentypen wie `int`, `double` oder `char` nicht aus, um die Daten zu repräsentieren, die wir benötigen.
- `struct`s erlauben uns, eigene Datentypen zu definieren, die mehrere Variablen enthalten können.

Zwei Beispiele: Eine Rationale Zahl als Bruch und ein Punkt im 2D-Raum.

# Anforderungen an unsere Datentypen

**Brüche**

- Wir wollen Brüche addieren, subtrahieren, multiplizieren und dividieren können.
- Wir wollen Brüche vergleichen können.
- Wir wollen sicherstellen, dass der Nenner nicht 0 ist.

**Punkte**

- Wir wollen zwei Punkte vergleichen können.
- Wir wollen Punkte, ähnlich wie Vektoren addieren und subtrahieren können.
- Wir wollen die Distanz zwischen zwei Punkten berechnen können.

**Beides**

- Wir wollen die Daten in unseren Datentypen lesen und schreiben können.

# Mögliche Implementierungen

**Brüche**

- Wir können zwei `int`s für Zähler und Nenner verwenden.
- Wir können eine `double` für den Wert des Bruchs verwenden.

**Punkte**

- Wir können die x- und y-Koordinaten verwenden.
- Wir können Polarkoordinaten verwenden.
- Wir könnten eine komplexe Zahl verwenden.[1]

[1] In C++ sind solche bereits definiert, für den Kurs aber nicht relevant.

# Mögliche Implementierungen

**Brüche**

- Wir können zwei `int`s für Zähler und Nenner verwenden.
- Wir können eine `double` für den Wert des Bruchs verwenden.

**Punkte**

- Wir können die x- und y-Koordinaten verwenden.
- Wir können Polarkoordinaten verwenden.
- Wir könnten eine komplexe Zahl verwenden.[1]

Spielt es eine Rolle, welche Implementierung wir wählen für den Benutzer? **Nein!**

[1] In C++ sind solche bereits definiert, für den Kurs aber nicht relevant.

# Mögliche Implementierungen

**Brüche**

- Wir können zwei `int`s für Zähler und Nenner verwenden.
- Wir können eine `double` für den Wert des Bruchs verwenden.

**Punkte**

- Wir können die x- und y-Koordinaten verwenden.
- Wir können Polarkoordinaten verwenden.
- Wir könnten eine komplexe Zahl verwenden.[1]

Spielt es eine Rolle, welche Implementierung wir wählen für den Benutzer? **Nein!**

OK, vielleicht doch ein bisschen... (Zusatzfrage dazu am Ende der Folien.)

[1] In C++ sind solche bereits definiert, für den Kurs aber nicht relevant.

# Definition der Structs

**Bruch**

```
struct Rational {
    int numerator;
    int denominator;
};
```

**Kartesischer Punkt**

```
struct Point {
    double x;
    double y;
};
```

**Polarer Punkt**

```
struct PolarPoint {
    double radius;
    double angle;
};
```

# Funktionen mit Structs

Structs können ganz normal als Parameter an Funktionen übergeben werden. Wir können auch Funktionen definieren, die Structs zurückgeben.

```
1  double Distance(Point p1, Point p2) {
2      double dx = p2.x - p1.x;
3      double dy = p2.y - p1.y;
4      return sqrt(dx*dx + dy*dy);
5  }
6
7  Point AddPoints(Point p1, Point p2) {
8      Point result;
9      result.x = p1.x + p2.x;
10     result.y = p1.y + p2.y;
11     return result;
12 }
```

50:50 Pub Tel

**Frage 2 ▪ 100 CHF**

```
void Shift(Point p) { p.x = p.x + 1; }
Point a; a.x = 2; a.y = 0; Shift(a);
```

Welchen Wert hat `a.x` danach?

A: 2

B: 3

C: Compilerfehler

D: Undefiniert

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 2 ▪ 100 CHF**

```
void Shift(Point p) { p.x = p.x + 1; }
Point a; a.x = 2; a.y = 0; Shift(a);
```

Welchen Wert hat `a.x` danach?

A: 2

B: 3

C: Compilerfehler

D: Undefiniert

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

# Operator Overloading

Cool wäre es, etwas wie `Point p3 = p1 + p2;` schreiben zu können. Dafür verwenden wir Operator Overloading.[2]

```
1  Point operator+(const Point& p1, const Point& p2) {
2      Point result;
3      result.x = p1.x + p2.x;
4      result.y = p1.y + p2.y;
5      return result;
6  }
```

**Zusammengefasst:** `operator+` ist einfach ein Funktionsname. `p1 + p2` ruft `operator+(p1, p2)` auf.

[2] In C++ sind Operatoren wie + für viele Typen bereits überladen. Deshalb könnt ihr z.B. strings addieren, genauso wie ints und doubles.

50:50 Pub Tel

**Frage 3 ▪ 200 CHF**

Warum übergeben wir die Punkte bei `operator+` als `const Point&`?

A: Es wird keine Kopie gemacht, die Argumente sind unveränderlich

B: Structs können sonst nicht Funktionen übergeben werden

C: Sonst lässt sich der Operator nicht überladen

D: Damit der Rückgabewert ebenfalls eine Referenz ist

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 3 ▪ 200 CHF**

Warum übergeben wir die Punkte bei `operator+` als `const Point&`?

A: Es wird keine Kopie gemacht, die Argumente sind unveränderlich

B: Structs können sonst nicht Funktionen übergeben werden

C: Sonst lässt sich der Operator nicht überladen

D: Damit der Rückgabewert ebenfalls eine Referenz ist

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 4 ▪ 300 CHF**

Welche der folgenden Dinge führt nicht zu einem Function Overload?

A: Unterschiedliche Anzahl an Parametern

B: Unterschiedliche Typen der Parameter

C: Nur ein unterschiedlicher Rückgabetyp

D: Gleiche Parametertypen in anderer Reihenfolge, z. B. (int, double) und (double, int)

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 4 ▪ 300 CHF**

Welche der folgenden Dinge führt nicht zu einem Function Overload?

A: Unterschiedliche Anzahl an Parametern

B: Unterschiedliche Typen der Parameter

C: Nur ein unterschiedlicher Rückgabetyp

D: Gleiche Parametertypen in anderer Reihenfolge, z. B. (int, double) und (double, int)

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

# Ein- und Ausgabe: Für die Interessierten

Für die Ein- und Ausgabe können wir den << und >> Operator überladen.

```
1 std::ostream& operator<<(std::ostream& os, const Point& p) {
2     os << "(" << p.x << ", " << p.y << ")";
3     return os;
4 }
```

```
1 std::istream& operator>>(std::istream& is, Point& p) {
2     is >> p.x >> p.y;
3     return is;
4 }
```

Bemerkung: Wir haben jetzt nur den kartesischen Punkt implementiert. Wir hätten aber auch genauso gut den polaren Punkt implementieren können. Für den Benutzer ist das egal, solange die Schnittstelle gleich bleibt.




Bildquelle: Orell Füssli

# Die custom Set Class

- Für diese Woche: `class` ist das Selbe wie `struct`.
- Für die Serien wird eine eigene Set-Klasse implementiert, die den mathematischen Mengenbegriff repräsentiert.
- Die Klasse besitzt folgende Funktionen:[3] Erstellen von Mengen, Aufrufen von Elementen mit `[]`, Vereinigung ($+$), Schneiden (-) und Kopieren von Mengen ($=$). Zusätzlich kann eingefügt werden mit `insert()` und die Grösse der Menge kann mit `size()` abgefragt werden. Die Elemente der Menge sind automatisch sortiert.
- Namen der Klasse: `CharSet` für Mengen von `char`s, `SetOfCharSets` für Mengen von `CharSets`.

[3] Die Implementierung der Klasse ist nicht Teil der Übung. Wie gesehen, interessiert uns nur was die Klasse kann, nicht wie sie es macht.

# Die Set-Klasse: Übersicht der Operationen

| **Operation** | **Syntax** | **Wirkung** |
|---|---|---|
| Erstellen | `CharSet s;` | Neue Menge |
| Zugriff | `s[i]` | Element an Position `i` (sortiert) |
| Vereinigung | `s1 + s2` | Neue Menge mit Elementen aus beiden |
| Schneiden | `s1 - s2` | Schnitt: nur Elemente, die in beiden vorkommen |
| Kopieren | `s1 = s2` | `s1` wird eine Kopie von `s2` |
| Einfügen | `s.insert(x)` | Fügt `x` in die Menge ein |
| Grösse | `s.size()` | Anzahl Elemente |

Die Elemente sind **automatisch sortiert**.

# Beispiel: `insert()`, `size()` und `[]`

**Code**

```
CharSet s;        // leere Menge
s.insert('e');
s.insert('b');
s.insert('h');
s.insert('b');

s.size();         // ?
s[0];             // ?
s[2];             // ?
```

# Beispiel: `insert()`, `size()` und `[]`

**Code**

```
CharSet s;        // leere Menge
s.insert('e');
s.insert('b');
s.insert('h');
s.insert('b');

s.size();         // ?
s[0];             // ?
s[2];             // ?
```

**Ergebnis**

- `s.size()` $\rightarrow$ 3

**Warum?**

1. Vier `insert()`-Aufrufe, aber das zweite `'b'` ist ein Duplikat und wird ignoriert $\Rightarrow$ Grösse 3.

# Beispiel: `insert()`, `size()` und `[]`

**Code**

```
CharSet s;        // leere Menge
s.insert('e');
s.insert('b');
s.insert('h');
s.insert('b');

s.size();         // ?
s[0];             // ?
s[2];             // ?
```

**Ergebnis**

- `s.size()` $\rightarrow$ 3
- `s[0]` $\rightarrow$ `'b'`

**Warum?**

1. Vier `insert()`-Aufrufe, aber das zweite `'b'` ist ein Duplikat und wird ignoriert $\Rightarrow$ Grösse 3.
2. Die Menge sortiert automatisch: $\{b, e, h\}$.

# Beispiel: `insert()`, `size()` und `[]`

**Code**

```
CharSet s;          // leere Menge
s.insert('e');
s.insert('b');
s.insert('h');
s.insert('b');

s.size();           // ?
s[0];               // ?
s[2];               // ?
```

**Ergebnis**

- `s.size()` $\rightarrow$ 3
- `s[0]` $\rightarrow$ `'b'`
- `s[2]` $\rightarrow$ `'h'`

**Warum?**

1. Vier `insert()`-Aufrufe, aber das zweite `'b'` ist ein Duplikat und wird ignoriert $\Rightarrow$ Grösse 3.
2. Die Menge sortiert automatisch: $\{b, e, h\}$.
3. `[]` zählt ab 0 in dieser sortierten Reihenfolge, nicht in Einfügereihenfolge.

50:50 Pub Tel

**Frage 5 ▪ 500 CHF**

Seien `x`= $\{a, c\}$ und `y`= $\{b, c\}$ zwei `CharSets`. Welchen Wert liefert `(x + y).size()`?

A: 4

B: 3

C: 2

D: Compilerfehler

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| **5** | **500 CHF** |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 5 ▪ 500 CHF**

Seien `x`= $\{a, c\}$ und `y`= $\{b, c\}$ zwei `CharSets`. Welchen Wert liefert `(x + y).size()`?

A: 4

B: 3

C: 2

D: Compilerfehler

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

# [code]expert: Power Set

- [code]expert: Power Set

# Lösungsstrategie

Lösen wir zuerst etwas einfaches: Was ist die Potenzmenge von $\{\}$ (Leere Menge)?

# Lösungsstrategie

Lösen wir zuerst etwas einfaches: Was ist die Potenzmenge von $\{\}$ (Leere Menge)?

$$\mathcal{P}(\{\}) = \{\{\}\}.$$

OK, etwas schwierieger: Was ist die Potenzmenge von einem Element $\{a\}$?

# Lösungsstrategie

Lösen wir zuerst etwas einfaches: Was ist die Potenzmenge von $\{\}$ (Leere Menge)?

$$\mathcal{P}(\{\}) = \{\{\}\}.$$

OK, etwas schwierieger: Was ist die Potenzmenge von einem Element $\{a\}$?

$$\mathcal{P}(\{a\}) = \{\{\}, \{a\}\}.$$

Immer noch einfach, was ist die Potenzmenge von zwei Elementen $\{a, b\}$?

# Lösungsstrategie

Lösen wir zuerst etwas einfaches: Was ist die Potenzmenge von $\{\}$ (Leere Menge)?

$$\mathcal{P}(\{\}) = \{\{\}\}.$$

OK, etwas schwierieger: Was ist die Potenzmenge von einem Element $\{a\}$?

$$\mathcal{P}(\{a\}) = \{\{\}, \{a\}\}.$$

Immer noch einfach, was ist die Potenzmenge von zwei Elementen $\{a, b\}$?

$$\mathcal{P}(\{a, b\}) = \{\{\}, \{a\}, \{b\}, \{a, b\}\}.$$

Wir bemerken: $\mathcal{P}(\{a\}) \subset \mathcal{P}(\{a, b\})$
Allgemeiner: $S \subset T \implies \mathcal{P}(S) \subset \mathcal{P}(T)$

# Lösungsstrategie

Wir wissen $S \subset T \implies \mathcal{P}(S) \subset \mathcal{P}(T)$. Wie können wir nun aus $\mathcal{P}(S)$ $\mathcal{P}(S \cup \{x\})$ machen?[4]

[4] Für die Mathematiker: $x \notin S$.

# Lösungsstrategie

Wir wissen $S \subset T \implies \mathcal{P}(S) \subset \mathcal{P}(T)$. Wie können wir nun aus $\mathcal{P}(S)$ $\mathcal{P}(S \cup \{x\})$ machen?[4]
Wir können $\mathcal{P}(S)$ nehmen und für jedes Element $A \in \mathcal{P}(S)$ ein neues Element $A \cup \{x\}$ erstellen.

$$\mathcal{P}(S \cup \{x\}) = \mathcal{P}(S) \cup \{A \cup \{x\} : A \in \mathcal{P}(S)\}.$$

Wir haben also die Potenzmenge von $S$ genommen und für jedes Element $A$ ein neues Element $A$ mit dem neuen Element $x$ erstellt.

Somit können wir also wie folgt vorgehen:

[4] Für die Mathematiker: $x \notin S$.

# Lösungsstrategie

Wir wissen $S \subset T \implies \mathcal{P}(S) \subset \mathcal{P}(T)$. Wie können wir nun aus $\mathcal{P}(S)$ $\mathcal{P}(S \cup \{x\})$ machen?[4]
Wir können $\mathcal{P}(S)$ nehmen und für jedes Element $A \in \mathcal{P}(S)$ ein neues Element $A \cup \{x\}$ erstellen.

$$\mathcal{P}(S \cup \{x\}) = \mathcal{P}(S) \cup \{A \cup \{x\} : A \in \mathcal{P}(S)\}.$$

Wir haben also die Potenzmenge von $S$ genommen und für jedes Element $A$ ein neues Element $A$ mit dem neuen Element $x$ erstellt.

Somit können wir also wie folgt vorgehen:

1. Wähle ein beliebiges Element $x \in S$.
2. Erstelle eine neue Menge $S' = S \setminus \{x\}$.
3. Berechne $\mathcal{P}(S')$.
4. Erstelle die Potenzmenge von $S$ mit der Formel oben.

Preisfrage: Wie berechnen wir $\mathcal{P}(S')$?

[4] Für die Mathematiker: $x \notin S$.

# Lösungsstrategie

Wir wissen $S \subset T \implies \mathcal{P}(S) \subset \mathcal{P}(T)$. Wie können wir nun aus $\mathcal{P}(S)$ $\mathcal{P}(S \cup \{x\})$ machen?[4]
Wir können $\mathcal{P}(S)$ nehmen und für jedes Element $A \in \mathcal{P}(S)$ ein neues Element $A \cup \{x\}$ erstellen.

$$\mathcal{P}(S \cup \{x\}) = \mathcal{P}(S) \cup \{A \cup \{x\} : A \in \mathcal{P}(S)\}.$$

Wir haben also die Potenzmenge von $S$ genommen und für jedes Element $A$ ein neues Element $A$ mit dem neuen Element $x$ erstellt.

Somit können wir also wie folgt vorgehen:

1. Wähle ein beliebiges Element $x \in S$.
2. Erstelle eine neue Menge $S' = S \setminus \{x\}$.
3. Berechne $\mathcal{P}(S')$.
4. Erstelle die Potenzmenge von $S$ mit der Formel oben.

Preisfrage: Wie berechnen wir $\mathcal{P}(S')$? **Rekursiv!**
2. Frage: Funktioniert der Algorithmus? Terminiert er?

[4] Für die Mathematiker: $x \notin S$.

# Lösungsstrategie

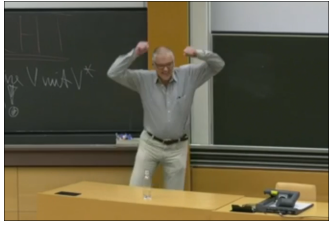
Wir wissen $S \subset T \implies \mathcal{P}(S) \subset \mathcal{P}(T)$. Wie können wir nun aus $\mathcal{P}(S)$ $\mathcal{P}(S \cup \{x\})$ machen?[4]
Wir können $\mathcal{P}(S)$ nehmen und für jedes Element $A \in \mathcal{P}(S)$ ein neues Element $A \cup \{x\}$ erstellen.

$$\mathcal{P}(S \cup \{x\}) = \mathcal{P}(S) \cup \{A \cup \{x\} : A \in \mathcal{P}(S)\}.$$

Wir haben also die Potenzmenge von $S$ genommen und für jedes Element $A$ ein neues Element $A$ mit dem neuen Element $x$ erstellt.

Somit können wir also wie folgt vorgehen:

1. Wähle ein beliebiges Element $x \in S$.
2. Erstelle eine neue Menge $S' = S \setminus \{x\}$.
3. Berechne $\mathcal{P}(S')$.
4. Erstelle die Potenzmenge von $S$ mit der Formel oben.



Preisfrage: Wie berechnen wir $\mathcal{P}(S')$? **Rekursiv!**
2. Frage: Funktioniert der Algorithmus? Terminiert er?
**NEIN!** Wir müssen eine Abbruchbedingung definieren.

[4] Für die Mathematiker: $x \notin S$.

50:50 Pub Tel

**Frage 6 ▪ 1 000 CHF**

Was ist $\mathcal{P}(\{\{\}\})$, also die Potenzmenge der Menge, die nur die leere Menge enthält?

A: $\{\{\}, \{\}\}$

B: $\{\{\}, \{\{\}\}\}$

C: $\{\{\{\}\}\}$

D: $\{\}$

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 6 ▪ 1 000 CHF**

Was ist $\mathcal{P}(\{\{\}\})$, also die Potenzmenge der Menge, die nur die leere Menge enthält?

A: $\{\{\}, \{\}\}$

B: $\{\{\}, \{\{\}\}\}$

C: $\{\{\{\}\}\}$

D: $\{\}$

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

# [code]expert: Power Set: Implementierung

- [code]expert: Power Set: Implementierung

# Zusammenfassung

## Structs

- Eigene Datentypen, die mehrere Variablen bündeln:

```
struct Point {
    double x;
    double y;
};
```

- Zugriff auf Member mit `p.x`

## Overloading

- **Function Overloading:** gleicher Name, aber andere Anzahl oder Typen der Parameter (nur der Rückgabetyp reicht **nicht**)
- **Operator Overloading:** `p1 + p2` ruft `operator+(p1, p2)` auf

# Mini-Frage 1 (Ja/Nein): Encapsulation

Wir tauschen die Implementierung von `Point` von kartesisch (`x`, `y`) zu polar (`radius`, `angle`).

Muss Code angepasst werden, der `Points` nur über `+` und `Distance` *benutzt*?

**Ja oder Nein?** Begründet eure Antwort.

# Mini-Frage 1: Lösung

**Antwort: Nein.**

- Der Aufrufer kennt nur die **Schnittstelle** (`+`, `Distance`), nicht die Darstellung der Daten. Das ist Encapsulation.
- **Warum nicht „Ja“?** „Ja“ wäre richtig, wenn der Code direkt `p.x` oder `p.y` lesen würde. Dann hängt er an der Implementierung.
- `operator+` und `Distance` selbst müssen neu geschrieben werden, denn sie gehören zur Implementierung.

**Merksatz:** Wer nur die Schnittstelle benutzt, ist von der Implementierung entkoppelt.

50:50 Pub Tel

**Frage 7 ▪ 2 000 CHF**

Wie viele rekursive Aufrufe werden benötigt, um die Potenzmenge einer Menge mit $n$ Elementen zu berechnen?

A: $2^n$

B: $n+1$

C: $n^2$

D: $n!$

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 7 ▪ 2 000 CHF**

Wie viele rekursive Aufrufe werden benötigt, um die Potenzmenge einer Menge mit $n$ Elementen zu berechnen?

A: $2^n$

B: $n+1$

C: $n^2$

D: $n!$

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

# Zusatzfrage 1: Lösung

**Richtig: B.**

1. Jeder Aufruf macht *einen* rekursiven Aufruf mit einer um 1 kleineren Menge.
2. Grössen: $n, n-1, \ldots, 1, 0$. Das sind $n+1$ Aufrufe.
3. Die Ausgabe wächst exponentiell, weil wir pro Schritt die bisherige Potenzmenge verdoppeln. Die Aufrufkette bleibt linear.

A verwechselt Ausgabegrösse und Aufrufanzahl, C und D passen nicht zu einem einzelnen Aufruf pro Ebene.

50:50 Pub Tel

**Frage 8 ▪ 4 000 CHF**

Was ist $\{a, b, c\} - \{b, c, d\}$ mit dem --Operator unserer `CharSet`-Klasse?

A: $\{a\}$

B: $\{b, c\}$

C: $\{a, d\}$

D: $\{a, b, c, d\}$

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| **8** | **4 000 CHF** |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

50:50 Pub Tel

**Frage 8 ▪ 4 000 CHF**

Was ist $\{a, b, c\} - \{b, c, d\}$ mit dem --Operator unserer `CharSet`-Klasse?

A: $\{a\}$

B: $\{b, c\}$

C: $\{a, d\}$

D: $\{a, b, c, d\}$

| | |
|---|---|
| 15 | 1 000 000 CHF |
| 14 | 500 000 CHF |
| 13 | 125 000 CHF |
| 12 | 64 000 CHF |
| 11 | 32 000 CHF |
| 10 | 16 000 CHF |
| 9 | 8 000 CHF |
| 8 | 4 000 CHF |
| 7 | 2 000 CHF |
| 6 | 1 000 CHF |
| 5 | 500 CHF |
| 4 | 300 CHF |
| 3 | 200 CHF |
| 2 | 100 CHF |
| 1 | 50 CHF |

# Zusatzfrage 2: Lösung

**Richtig: B.** – ist bei `CharSet` der **Schnitt**: nur Elemente, die in beiden Mengen vorkommen.

- **A falsch:** Das wäre die Differenz $\{a, b, c\} \setminus \{b, c, d\}$. Die Falle, wenn man bei „$-$“ an Mathematik denkt.
- **C falsch:** Das ist die symmetrische Differenz.
- **D falsch:** Das ist die Vereinigung, also +.

# Tipps zu den Übungen

**Set Product / All Permutations**

# Tipps zu den Übungen

**Set Product / All Permutations**

- Sind eher schwierige Aufgaben, die ihr nicht unbedingt machen müsst meiner Meinung nach.
- Verwendet die Operationen der Set-Klasse, die wir gesehen haben.